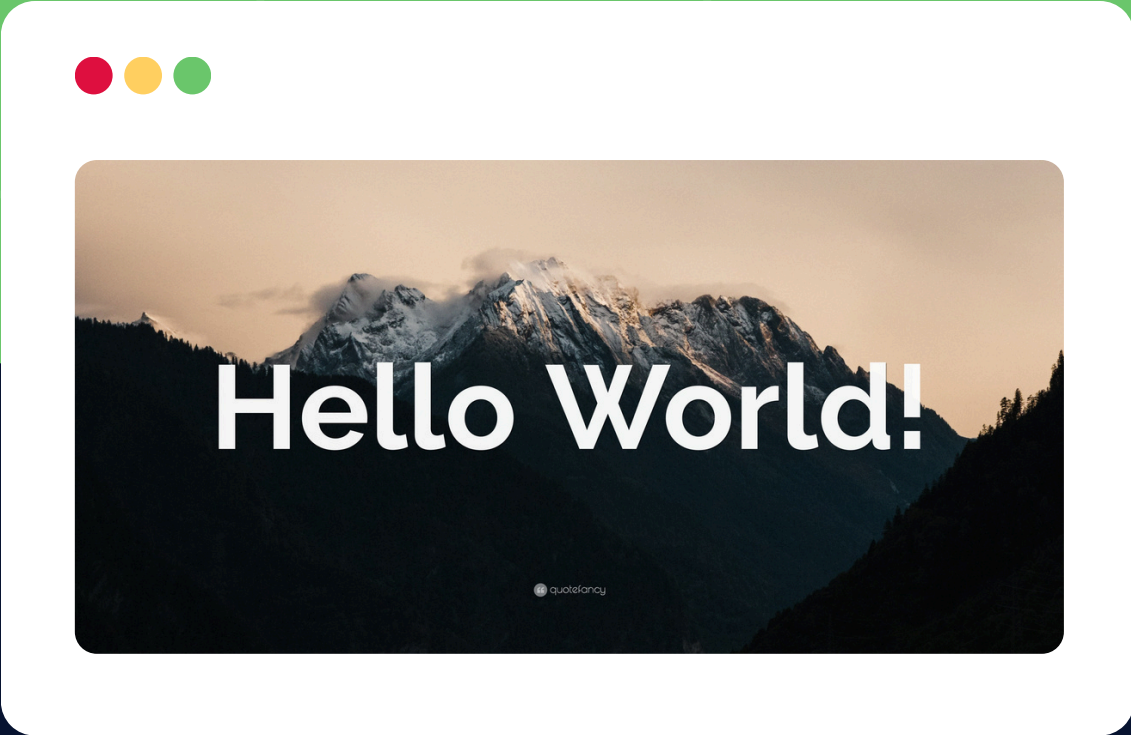




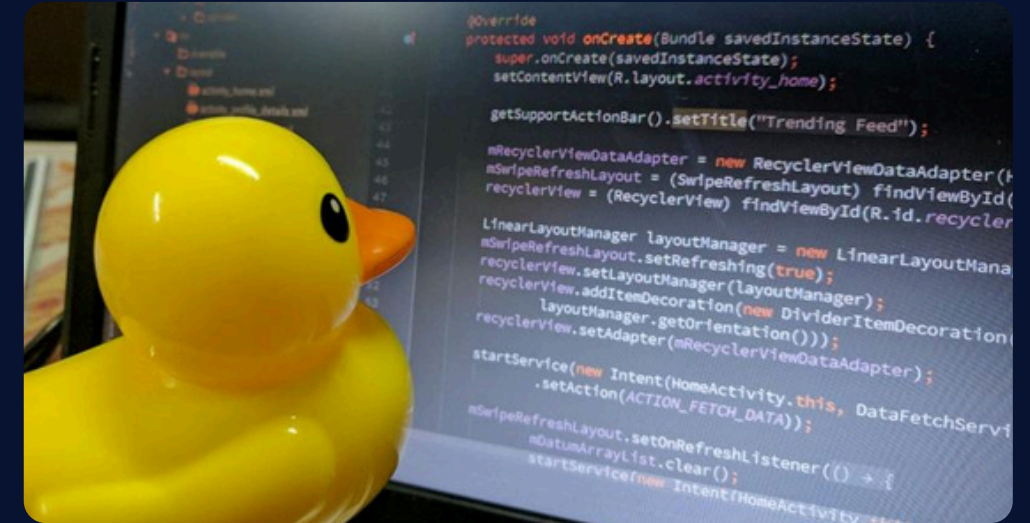
Hello World!



Data Preprocessing & Feature Engineering



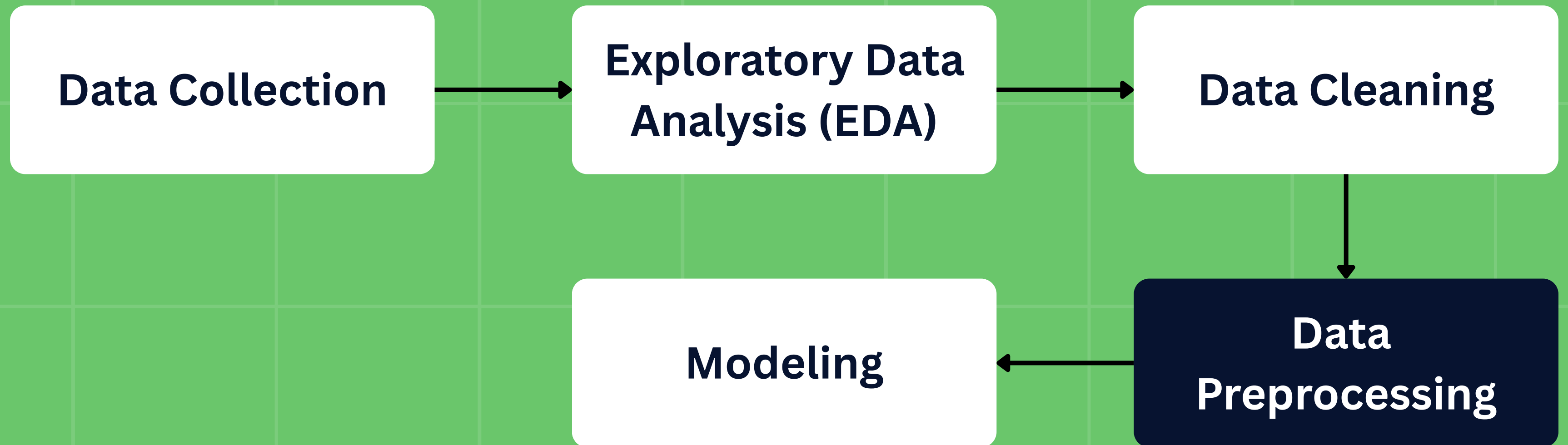
Session -> 6





Data Preprocessing

Data Preparation Cycle for Modeling



Data Preparation Cycle for Modeling

- 1- Data Collection:** Collect data that is relevant and suitable for the problem you want to solve.
- 2- EDA:** Explore the data to understand its patterns, relationships, and any issues that need attention.
- 3-Data Cleaning:** Fix the problems identified during EDA—such as missing values, inconsistencies, or errors—to make the dataset complete and reliable.

What is Data Preprocessing?

- After ensuring that the data is complete, consistent, and valid, we still cannot directly feed it into a machine learning model. The data must be transformed into a suitable format that the model can interpret and learn from. This step is called **data preprocessing**.
- In this section, we will learn how to preprocess **categorical** and **numerical data**:

What is Data Preprocessing?

- **Handling Categorical Data:**
 - Label Encoding
 - One-Hot Encoding
- **Handling Numerical Data:**
 - Normalization
 - Standardization / Scaling

Handling Categorical Data

Label Encoding

- Label Encoding is converting categories into numerical labels (assigns a unique number to each category)
- Label encoding is preferred for categories that are **ordered**, such as:
 - Grades (A+, A, A-, B,...)
 - Age groups (Child, Teenager, Adult)
 - Sizes (small, medium, large,...)
- Label encoding is also preferred when there are many categories, even if it has no order.

Handling Categorical Data

Label Encoding

	T-shirt ID	Size	Size_Encoded
0	101	Small	0
1	202	Medium	1
2	303	Small	0
3	404	Large	2
4	505	Large	2

Here we encoded small-> 0, med-> 1, and large-> 2

Handling Categorical Data

Label Encoding

1- Label Encoding using scikit learn:

```
from sklearn.preprocessing import LabelEncoder  
  
le = LabelEncoder()  
df['size_encoded'] = le.fit_transform(df['size'])
```

2- Label Encoding using map:

```
size_map = {'small': 0, 'medium': 1, 'large': 2}  
df['size_encoded'] = df['size'].map(size_map)
```

Handling Categorical Data

One-Hot Encoding

- One-hot encoding creates separate binary columns to represent each category.
- It is preferred when the categories **have no natural order**, such as:
 - Colors (red, blue, yellow, ...)
 - Blood types (A, B, AB, O)
 - Job titles (HR, IT, PR, Data Analyst)
- One-hot encoding is most suitable when the number of categories is relatively small, to avoid high dimensionality.

Handling Categorical Data

One-Hot Encoding

	Item ID	Color	Color_Blue	Color_Green	Color_Red	Color_Yellow
0	101	Red	False	False	True	False
1	202	Blue	True	False	False	False
2	303	Red	False	False	True	False
3	404	Yellow	False	False	False	True
4	505	Green	False	True	False	False

Handling Categorical Data

One-Hot Encoding

1- One-hot Encoding using scikit learn:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(sparse_output=False)
encoded_array = encoder.fit_transform(df[['Color']])

# Convert the encoded array to a DataFrame with meaningful column names
encoded_df = pd.DataFrame(encoded_array, columns=encoder.get_feature_names_out(['Color']))

# Concatenate with original DataFrame (dropping 'Color')
df = pd.concat([df.drop('Color', axis=1), encoded_df], axis=1)
```

Handling Categorical Data

One-Hot Encoding

2- One-hot Encoding using get dummies:

```
import pandas as pd
```

```
df_encoded = pd.get_dummies(df, columns=['Color'])
```

Handling Numerical Data

- Feature scaling is an important step in preparing data for machine learning models. It involves transforming the values of features in a dataset to a similar scale, ensuring that all features **contribute equally** to the model's learning process.
- There are two methods for feature scaling:
 - Normalization
 - Standardization

Handling Numerical Data

Normalization (Min-Max normalization)

- Normalization scales data to a range (usually 0 to 1) based on minimum and maximum values.

$$\mathbf{X_normalized} = \frac{\mathbf{X - Min(X)}}{\mathbf{Min(X) - Max(X)}}$$

Handling Numerical Data

Normalization (Min-Max normalization)

```
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
```

Handling Numerical Data

Standardization

- Standardization transforms the data in such a way that it will have a mean equal to zero and a standard deviation of 1.

$$\mathbf{X_standardized} = \frac{\mathbf{X - Mean(X)}}{\mathbf{Standard_Deviation}}$$

Handling Numerical Data

Standardization

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])
```

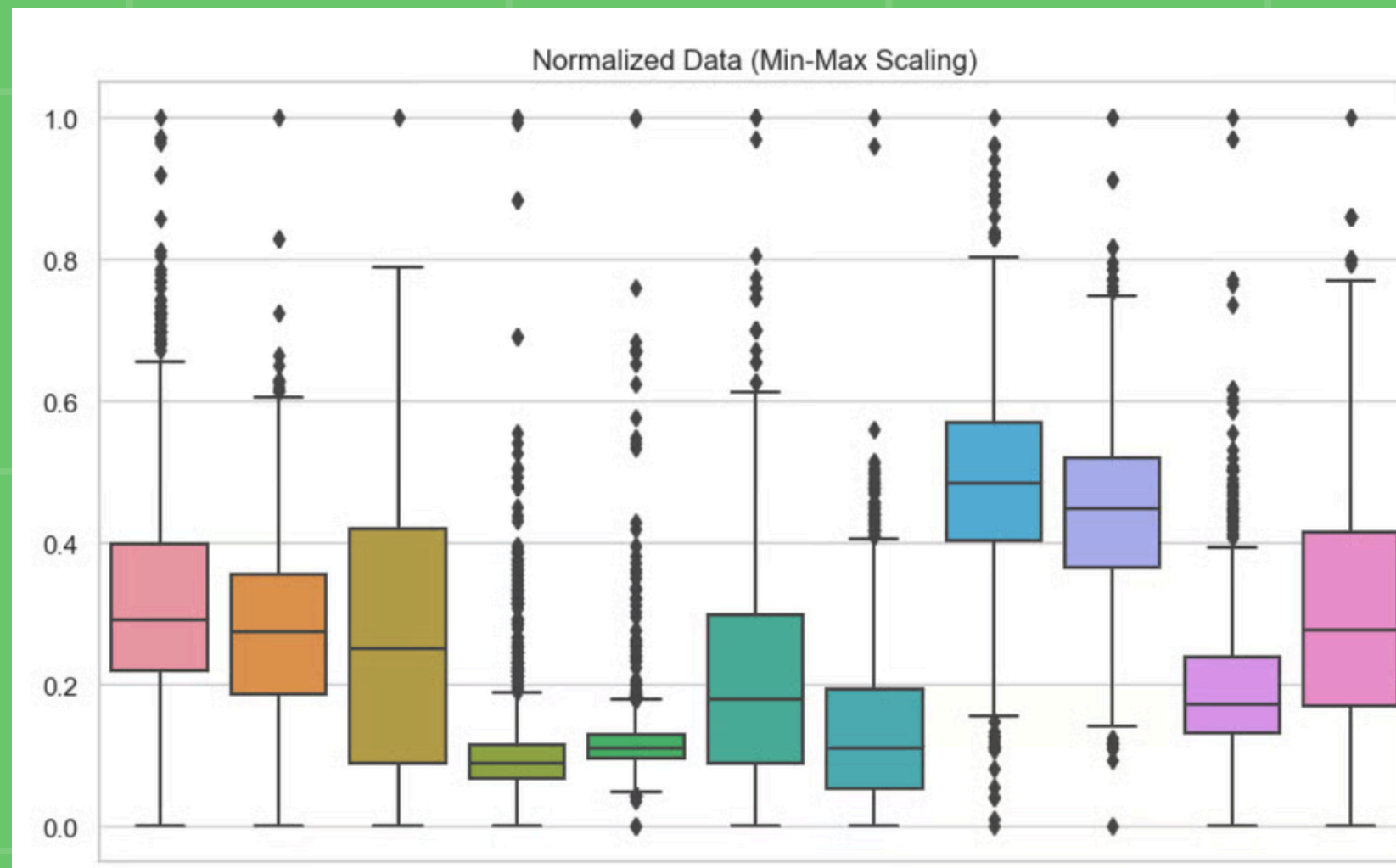
Handling Numerical Data

Standrization VS Normalization

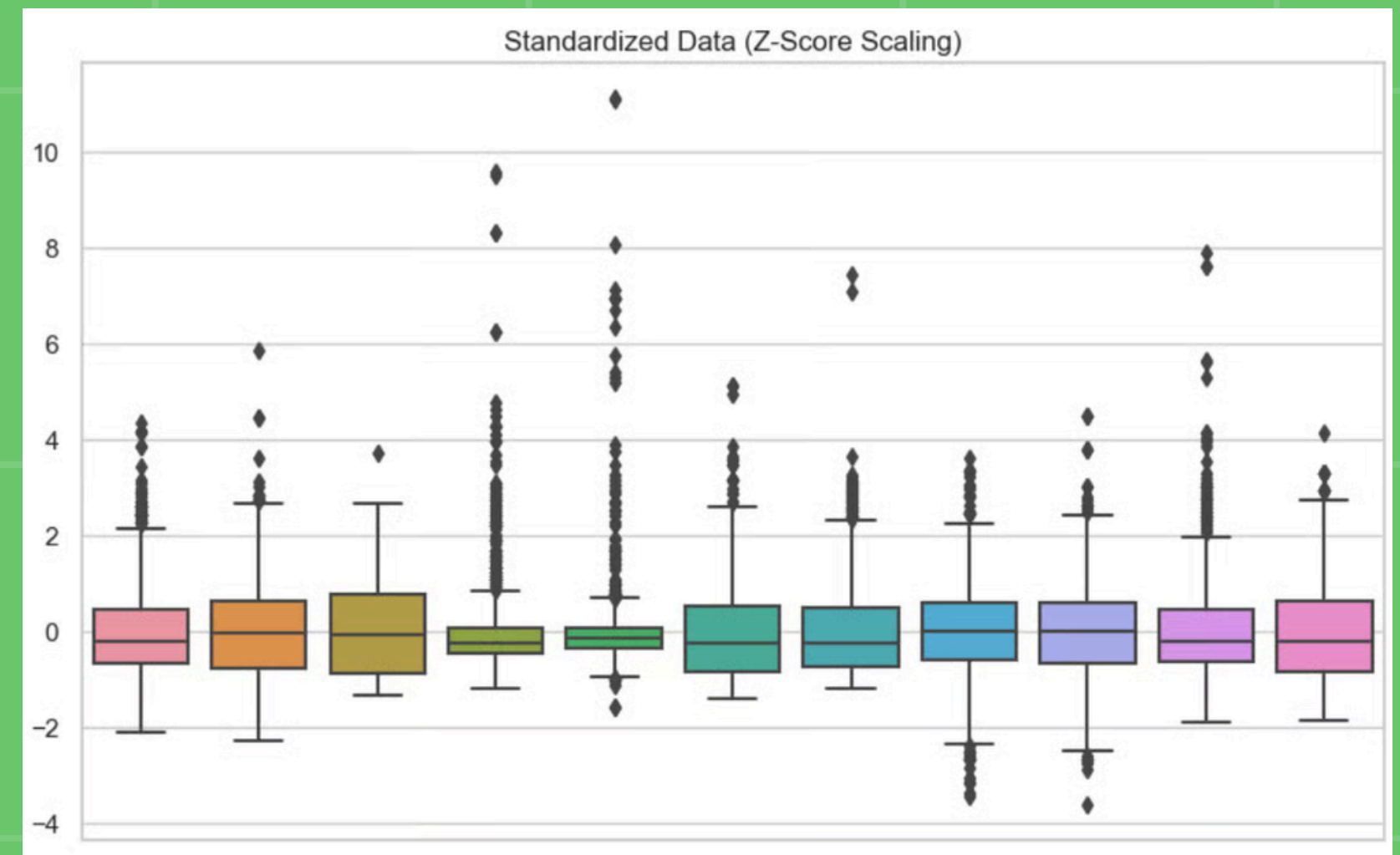
Aspect	Normalization	Standrization
Definition	Rescales data proportionally to fit within a fixed range	Transforms data so it has a mean of 0 and a standard deviation of 1.
Range	0 to 1	Unbounded; values usually fall around -3 to +3.
When to use	Best when you need data in a fixed range.	• Best when the algorithm assumes normally distributed data • Works well when data contains outliers.

Handling Numerical Data

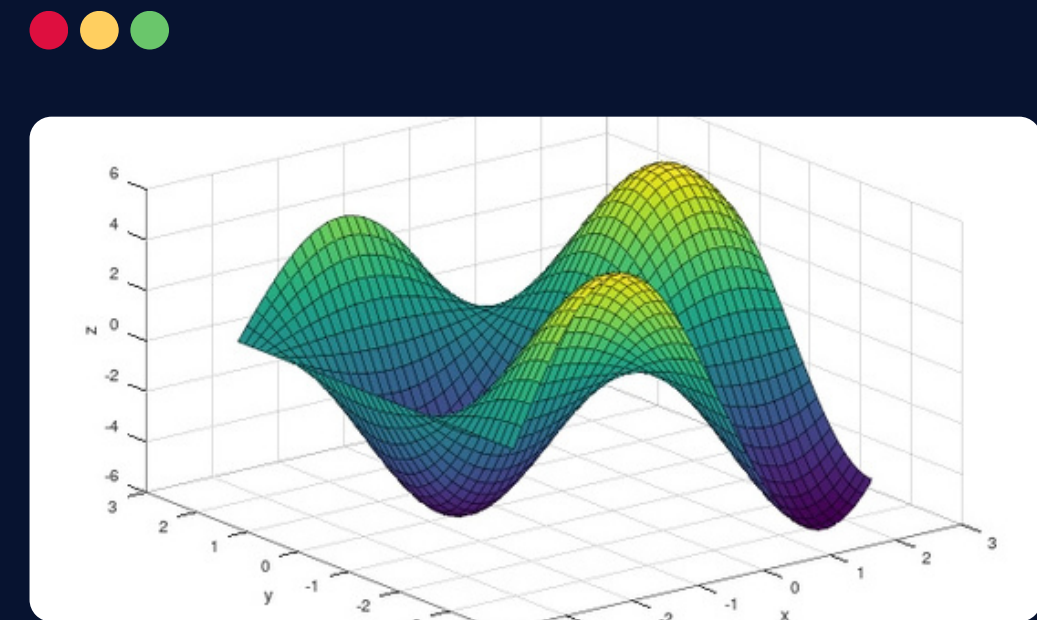
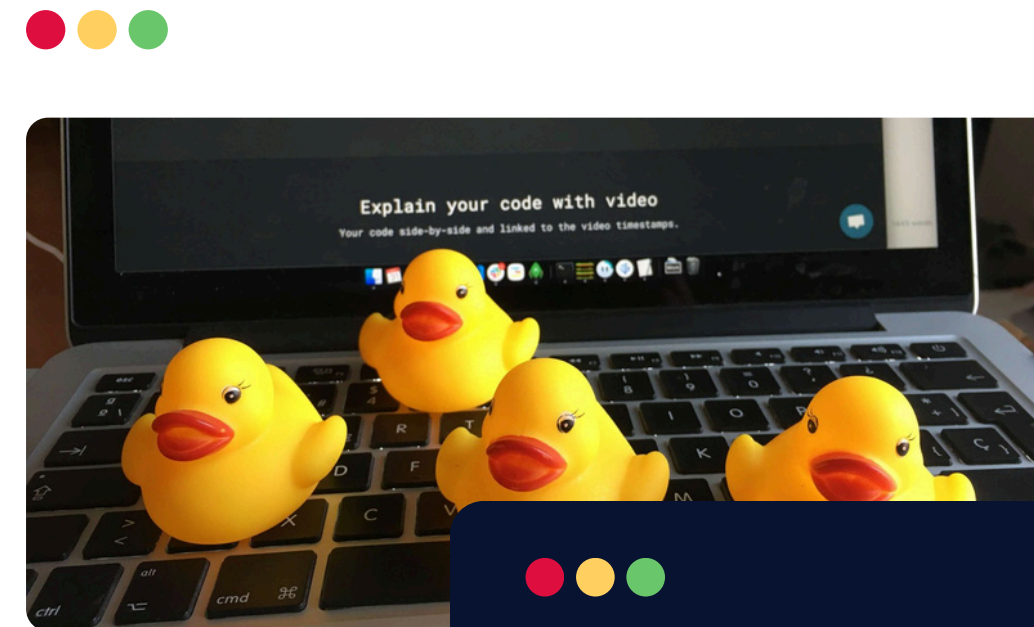
Standardization VS Normalization



Normalized data



Standardized data



Thank You

created by:
Ahd Ehab